

RELATÓRIO TRABALHO PRÁTICO – 2ª FASE

Diogo Alexandre Alves da Silva nº 31504

UC: Programação Orientada a Objetos

Professor: Luís Ferreira

Novembro, 2025

Indice

Glossário e Siglas ordenado por ordem alfabética.....	6
Resumo	7
Introdução	7
Motivação.....	7
Enquadramento	7
Estado da Arte	8
Conceitos / Fundamentos Teóricos	8
C#.....	8
Programação Orientada a Objetos.....	8
Classes.....	8
Abstração	8
Problema	9
Abordagem.....	9
Implementação	9
Desenvolvimento	10
1ª Fase	10
Classes.....	10
Pessoas.cs	10
Pessoa.....	10
Resumo	10
Atributos	10
Métodos principais.....	10
Overrides.....	10
Operadores	10
Funcionario	10
Resumo	10
Atributos	10
Métodos principais.....	10
Overrides.....	10
Operadores	10
Paciente	10
Resumo	10
Atributos	10
Métodos principais.....	11

Diogo Alexandre Alves Silva nº31504 Projeto-1ª Fase

Overrides.....	11
Medico	11
Resumo	11
Atributos	11
Métodos principais.....	11
Overrides.....	11
Enfermeiro	11
Resumo	11
Atributos	11
Métodos principais.....	11
Overrides.....	11
Auxiliar	11
Resumo	11
Atributos	11
Métodos principais.....	11
Overrides.....	11
Consultas.cs	11
Consulta	11
Resumo	11
Atributos	11
Métodos principais.....	11
Overrides.....	11
Operadores	11
Diagnostico	11
Resumo	11
Atributos	12
Métodos principais.....	12
Overrides.....	12
Exame.....	12
Resumo	12
Atributos	12
Métodos principais.....	12
Overrides.....	12
Operadores	12
ResultadoExame.....	12
Resumo	12

Diogo Alexandre Alves Silva nº31504 Projeto-1ª Fase

Atributos	12
Métodos principais	12
Overrides	12
InternamentoHospital.cs	12
Quarto	12
Resumo	12
Atributos	12
Métodos principais	12
Overrides	12
Operadores	12
Cama	12
Resumo	12
Atributos	12
Métodos principais	12
Overrides	12
Operadores	12
InternamentoHospital	12
Resumo	12
Atributos	13
Métodos principais	13
Overrides	13
Operadores	13
EnfermagemCuidados	13
Resumo	13
Atributos	13
Métodos	13
Overrides	13
Operadores	13
2ª Fase	13
Estrutura N-TIER	13
1. A camada BO (Business Objects):	13
2. Camada Dados	14
3. Camada Regras	14
4. Camada Programa	14
DRY	14
SOLID	14

Diogo Alexandre Alves Silva nº31504 Projeto-1ª Fase

Estruturas de dados	15
Tipos de Estruturas.....	15
Interfaces.....	16
ICrud<T,TId>	16
Aplicação no Projeto.....	16
IComparable<T>.....	16
Persistência.....	16
Implementação	17
Testes Unitários.....	17
Testes Implementados	17
Validação de NIF (4 testes)	17
Gestor de Pacientes (4 testes).....	17
Funcionalidade Geral	18
Fluxo de Funcionamento do Sistema	18
Repositório GitHub:	19
Conclusão.....	19
Referências.....	20
C# : Wikipédia, 15-11-2025	20
Abstração :Rocketseat, 2025	20
Classe: Wikipédia, 2025.....	20
Programação orientada a objetos: Wikipédia 2025	20
SOLID: Medium,2025	20

Índice de figuras

Figura 1 - Diagrama de Classes do projeto[1]	9
Figura 2 Estrutura Básica do N-TIER do projeto	13

Glossário e Siglas ordenado por ordem alfabética

POO – Programação Orientada a Objetos

UC- Unidade Curricular

txt- Texto

Doxygen- Ferramenta que gera documentação automática do código

Resumo

Este trabalho foi desenvolvido no âmbito da *UC* de *POO*, com o objetivo de aplicar e consolidar conceitos de objetos, nomeadamente classes, atributos e métodos e os pilares de *POO*, como herança, polimorfismo, abstração e encapsulamento.

O problema proposto consiste na criação de um sistema que permite a gestão de um hospital.

O projeto produziu a criação de uma biblioteca em *C#*, que possui diferentes classes, com os seus atributos e métodos que permitem a implementação de uma gestão de um Centro de Saúde. Também para melhorar a compreensibilidade do projeto desenvolveu-se uma documentação no código.

Introdução

O presente capítulo pretende contextualizar o trabalho realizado ao apresentar a motivação para o seu desenvolvimento, o seu enquadramento e os principais objetivos.

Motivação

No contexto deste projeto foi proposto o desenvolvimento de uma biblioteca *C#* que permita gerir as operações básicas de um centro de saúde: marcação de consultas, registo de diagnósticos, prescrições, exames, internamentos e cuidados de enfermagem. A resolução deste problema permitiu aplicar, de forma prática e consolidada, os pilares da *POO* – encapsulamento, herança, polimorfismo e abstração – sem depender de estruturas de dados complexas, focando-se antes na modelação correta de classes, na relação entre elas e na organização modular do código, aspetos essenciais no curso.

Enquadramento

Este projeto foi desenvolvido no âmbito da unidade curricular de Programação Orientada a Objetos (*POO*), integrada no curso de Licenciatura em Engenharia Informática, sob orientação do Professor Luís Ferreira. Trata-se de um trabalho prático individual, inserido na 1.ª fase de avaliação da unidade curricular, com o objetivo de aplicar e consolidar os conhecimentos adquiridos sobre os princípios fundamentais da programação orientada a objetos.

O trabalho consistiu na implementação de uma biblioteca em *C#*, que modela um sistema de gestão de um centro de saúde, permitindo a realização de operações como marcação de consultas, registo de diagnósticos, exames, internamentos e cuidados de enfermagem. O desenvolvimento focou-se na aplicação dos pilares da *POO*, encapsulamento, herança, polimorfismo e abstração, bem como na organização modular do código e na documentação técnica.

Estado da Arte

O presente capítulo pretende contextualizar as ferramentas e estruturas utilizadas ao longo do projeto para realizar o trabalho proposto.

Conceitos / Fundamentos Teóricos

C#

“C# é uma linguagem de programação, multiparadigma, de tipagem forte, desenvolvida pela Microsoft como parte da plataforma .NET. A sua sintaxe orientada a objetos foi baseada no C++ mas inclui muitas influências de outras linguagens de programação, como Object Pascal e, principalmente, Java. O código fonte é compilado para Common Intermediate Language (CIL) que é interpretado pela máquina virtual Common Language Runtime (CLR).” (Referência: C# : [Wikipédia, 15-11-2025](#))

manutenção em diversas linguagens de programação e projetos escalas.” (Referência traduzida com Google Tradutor: Abstração :Rocketseat, 2025)

Programação Orientada a Objetos

“Programação orientada a objetos (POO, ou OOP segundo as suas siglas em inglês) é um paradigma de programação baseado no conceito de "objetos", que podem conter dados na forma de campos, também conhecidos como atributos, e códigos, na forma de procedimentos, também conhecidos como métodos.” (Referência: Programação orientada a objetos: Wikipédia 2025)

Classes

“uma classe é um Tipo abstrato de Dados (TAD); ou seja, uma descrição que abstrai um conjunto de objetos com características similares (um projeto do objeto), é um código da linguagem de programação orientada a objetos que define e implementa um novo tipo de objeto, que terão características (atributos) que guardaram valores e, também funções específicas para manipular estes.” (Referência: Classe: [Wikipédia, 2025](#))

Abstração

“A abstração é um princípio fundamental da Programação Orientada a Objetos que envolve a identificação e a modelagem das características e comportamentos essenciais de um objeto, ignorando os detalhes irrelevantes ou secundários para o contexto em questão. Em essência, a abstração permite aos desenvolvedores criar modelos simplificados de entidades complexas do mundo real, focando apenas nos aspectos que são importantes para a aplicação sendo desenvolvida.” (Referência: Abstração :Rocketseat, 2025)

Problema

O problema apresentado foi o seguinte: Criar um sistema que faça a gestão de um hospital (Mas na primeira fase, foram feitas as classes, os fundamentos do resto)

Abordagem

A abordagem escolhida para a resolução do problema foi a implementação de classes bem compostas como se verá posteriormente. Essa escolha deve-se à sua eficiência e simplicidade frente às exigências do projeto nas funções. Caso as classes estejam bem definidas o resto do projeto será mais fácil pois a base estará bem preparada para a carga que for colocada em cima.

Implementação

Para iniciar este projeto projetou-se as classes necessárias para o sistema anteriormente referido.

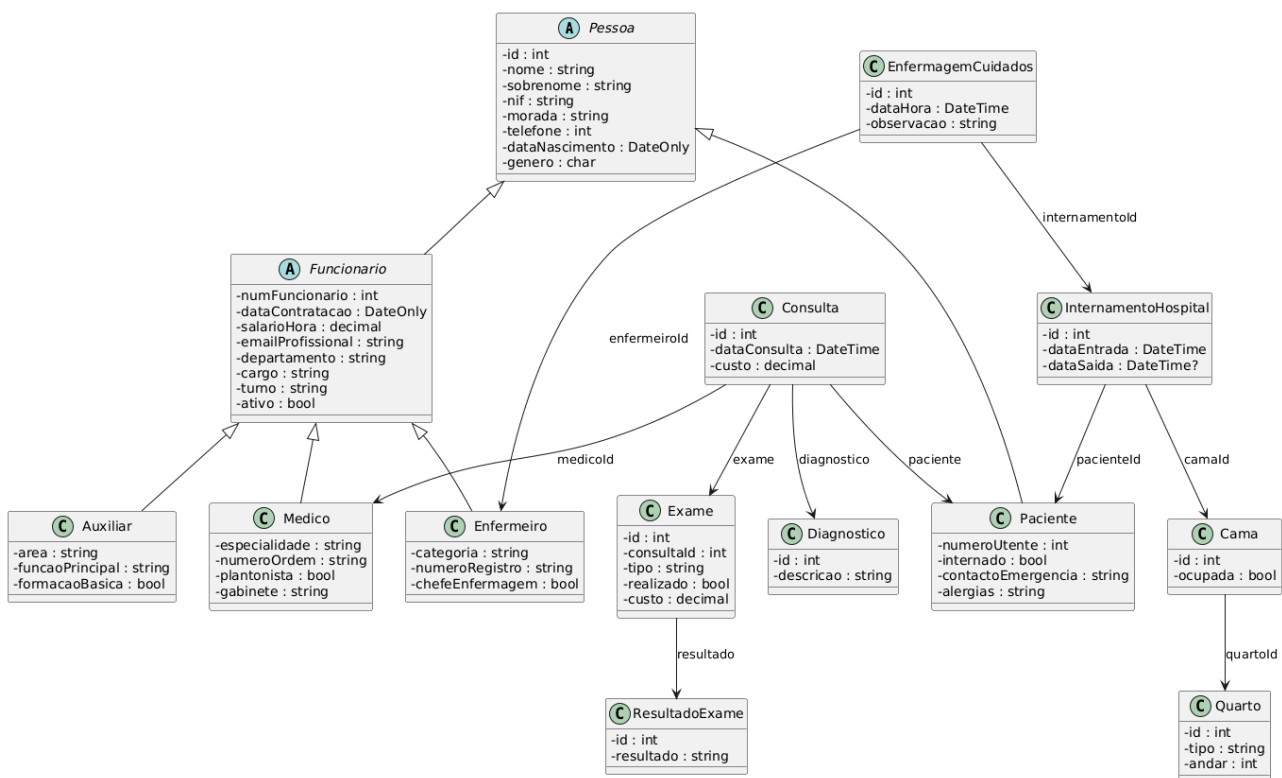


Figura 1 - Diagrama de Classes do projeto[1]

Como se pode observar na Figura 1 - Diagrama de Classes do projeto, todas as classes encontram-se conectadas para poderem realizar o seu propósito de servir como meio para alguém gerir um hospital para isso procede-se as suas funcionalidades e conexões.

Desenvolvimento

O presente capítulo pretende mostrar as soluções e o desenvolvimento ao longo do projeto.

Para iniciar, este projeto teve de decidir como iria definir as regras/instruções de utilização do programa sendo elas:

- O hospital apenas atende clientes com ficha (ou em caso de urgência realizar a ficha depois)
- Todas as relações são únicas pois ainda não foram abordadas estruturas de dados
- Como o projeto ainda se encontra na primeira fase encontra-se problemas de implementação pelo facto de não ser capaz de se inserir estruturas de dados, então desenvolveu-se apenas a estrutura básica
- O projeto ficou com meia implementação de devolução de códigos de erros que será mais aprofundada na segunda fase

1ª Fase

Classes

Pessoas.cs

Pessoa

Resumo: Classe abstrata base para todas as pessoas do hospital.

Atributos: id, nome, sobrenome, nif, morada, telefone, dataNascimento, genero.

Métodos principais: NomeCompleto, calculaIdade, maiorIdade, ValidarNIF, GeneroExtenso.

Overrides: ToString(), GetTipo() (abstrato).

Operadores: ==, !=.

Funcionario

Resumo: Classe abstrata base para funcionários.

Atributos: numFuncionario, dataContratacao, salarioHora, emailProfissional, departamento, cargo, turno, ativo.

Métodos principais: ObterAnosServico, AumentarSalario.

Overrides: ToString(), GetTipo().

Operadores: ==, !=.

Paciente

Resumo: Representa um paciente.

Atributos: numeroUtente, internado, contactoEmergencia, alergias.

Diogo Alexandre Alves Silva nº31504 Projeto-1ª Fase

Métodos principais: AdicionarAlergia, RemoverAlergia, ObterUltimaConsulta, AptoAAalta, DarAlta.

Overrides: GetTipo().

Medico

Resumo: Representa um médico do hospital.

Atributos: especialidade, numeroOrdem, fazUrgencias, gabinete.

Métodos principais: EstaDisponivel, ObterPacientesDoDia, AdicionarEspecialidade, ConsultasMedico, AdicionarCuidadosEnfermeiro, RemoverCuidadosEnfermeiro.

Overrides: ToString(), GetTipo().

Enfermeiro

Resumo: Representa um enfermeiro.

Atributos: categoria, chefeEnfermagem.

Métodos principais: PodeSerChefe, CuidadosEnfermeiro, AdicionarCuidadosEnfermeiro, RemoverCuidadosEnfermeiro.

Overrides: ToString(), GetTipo().

Auxiliar

Resumo: Representa funcionário auxiliar (limpeza, TI, etc.).

Atributos: area, funcaoPrincipal.

Métodos principais: —

Overrides: ToString(), GetTipo().

Consultas.cs

Consulta

Resumo: Representa uma consulta médica, ligando paciente, médico, exames, diagnósticos e custo associado.

Atributos: id, paciente, medicoId, dataConsulta, exame, diagnostico, custo.

Métodos principais: AdicionarExame, RemoverExame, ListarExames, AdicionarDiagnostico, RemoverDiagnostico, ListarDiagnostico, CalcularCustoTotal.

Overrides: ToString().

Operadores: ==, !=.

Diagnostico

Resumo: Guarda um diagnóstico associado a uma consulta.

Atributos: id, descricao.

Métodos principais: —

Overrides: ToString().

Exame

Resumo: Representa um exame solicitado numa consulta.

Atributos: id, consultaId, tipo, resultado, realizado, custo.

Métodos principais: —

Overrides: ToString().

Operadores: ==, !=.

ResultadoExame

Resumo: Representa o resultado de um exame.

Atributos: id, resultado.

Métodos principais: —

Overrides: ToString().

InternamentoHospital.cs

Quarto

Resumo: Representa um quarto do hospital.

Atributos: id, tipo, andar.

Métodos principais: AdicionarCama, RemoverCama.

Overrides: ToString().

Operadores: ==, !=.

Cama

Resumo: Representa uma cama num quarto.

Atributos: id, quartoId, ocupada.

Métodos principais: —

Overrides: ToString().

Operadores: ==, !=.

InternamentoHospital

Resumo: Representa um internamento associando paciente, cama e datas.

Atributos: id, pacienteId, camaId, dataEntrada, dataSaida.

Métodos principais: —

Overrides: ToString().

Operadores: ==, !=.

EnfermagemCuidados

Resumo: Guarda observações feitas por um enfermeiro a um paciente internado.

Atributos: id, internamentoId, enfermeiroId, dataHora, observacao.

Métodos principais: —

Overrides: ToString().

Operadores: ==, !=.

2ª Fase

Nesta fase do trabalho foi realizada uma aprofundação nos conceitos de POO, nos conceitos de programação em C# em geral, como os conceitos de SOLID, DRY e estrutura N-TIER.

Estrutura N-TIER

A arquitetura N-Tier separa uma aplicação em camadas distintas, cada uma com responsabilidades específicas, o que leva à separação de responsabilidades, facilidade de manutenção e testabilidade.

Nesta segunda fase decidiu-se implementar uma estrutura básica onde

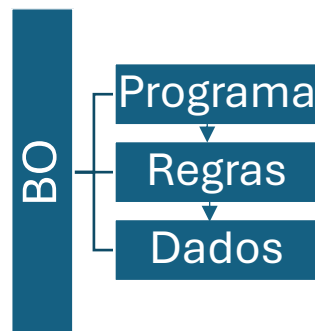


Figura 2 Estrutura Básica do N-TIER do projeto

Como se observa na *Figura 2 Estrutura Básica do N-TIER do projeto*, o projeto possui as classes bases (BO), que é referenciado no Programa, nas Regras e nos Dados.

1. A camada BO (Business Objects):

- São as classes criadas na 1ª Fase
- Representa as entidades do sistema (Paciente, Medico, Consulta, etc.)
- Sem dependências de outras camadas

2. Camada Dados

- Gere coleções de entidades (listas, dicionários)
- Implementa operações CRUD (Create, Read, Update, Delete)

3. Camada Regras

- Contém as regras de negócio do sistema
- Valida operações antes de persistir dados
- Exemplo: PacientesRegras verifica se um paciente tem +18 anos antes de registar
- Comunica apenas com a camada Dados

4. Camada Programa

- Interface com o utilizador (consola ou UI)
- Comunica apenas com a camada Regras
- Não contém lógica de negócio

DRY

O conceito de DRY, significa *DONT REPEAT YOURSELF*, como o próprio nome indica, não repetir código, ou seja, quando um código estiver a ser repetido extensivamente (sem ser uma linha básica), seria melhor criar uma função para utilizar quando esse código fosse necessário.

Neste projeto, por exemplo, pode-se encontrar este conceito no projeto Validacoes que possui a classe “Validacoes.cs”, onde o projeto possui as duas funções:

- `public static bool NifValido(string nif)`
- `public static bool EmailValido(string email)`

Estas funções seriam código repetido constantemente, assim rompendo o conceito DRY, fazendo esta pequena mudança, facilita a manutenção, a leitura e manipulação do código.

SOLID

SOLID é um acrónimo criado por Michael Feathers, após observar que cinco princípios da orientação a objetos e design de código (SOLID: Medium,2025)

- S- Princípio da Responsabilidade Única
 - Uma classe deve ter um, e somente um, motivo para mudar.
- O- Princípio Aberto-Fechado
 - Objetos ou entidades devem estar abertos para extensão, mas fechados para modificação,
- L- Princípio da substituição de Liskov

Diogo Alexandre Alves Silva nº31504 Projeto-1ª Fase

- Uma classe derivada deve ser substituível por sua classe base
- Princípio da Segregação da Interface
 - Uma classe não deve ser forçada a implementar interfaces e métodos que não irão utilizar
- D- Princípio da inversão da dependência
 - Dependenda de abstrações e não de implementações

Para todo este projeto tentou-se seguir este principio essencial de POO, ao longo do projeto, verifica-se todas essas questões, o **S** encontra-se em todas as classes, o **O** verifica-se por exemplo na classe Enfermeiro (*Enfermeiro.cs*) onde existe a função PodeSerChefe(string esp) que estende o comportamento de Enfermeiro. Na a letra **L**, encontra-se exemplos em todas as classes que herdam de outra, por exemplo, Paciente com Pessoa, todas as entidades Paciente podem ser tratadas como Pessoa. Na letra **I** verifica-se no projeto Interfaces que utiliza por exemplo o ICrud<T, Tid> para que todas as classes que utilizem possuem com necessidade (crud para todas as regras). Por fim a letra **D** está também nas Interfaces, onde utiliza-se a interface *IGestorPacienteBD.cs* para ligar GestorPacienteBD e PacientesRegras.

Estruturas de dados

Na 1ªFase apenas utilizou-se classes e apenas ligações 1:1, o que criou uma limitação onde um Paciente poderia apenas possuir uma Consulta, o que impossibilitaria verificar históricos. Para a 2ª Fase do projeto, identificaram-se ligações 1:1 que deveriam ser 1:M, para isso criaram-se Classes auxiliares no projeto Dados, ou seja, por exemplo uma classe Paciente passou para PacienteBD. Sendo esta última classe pertencente á camada Dados. Isto para não misturar responsabilidades e não quebrar o SOLID nem o DRY:

Paciente:

..... (Atributos)

Consulta consulta;

InternamentoHospital internamento;

.....

PacienteBD:

Paciente paciente;

Consultas consultas; (classe de dados que gere a estrutura de consultas)

InternamentosHospital internamentos; (classe que gere a estrutura de internamentos)

Tipos de Estruturas

Nesta secção falar-se-á da escolha das estruturas de dados que foram escolhidas.

Para quase todas as classes, utiliza-se uma List<T>, uma coleção genérica simples e eficaz de usar, em que não daria problemas nenhuns em utilizar pois as listas iriam agregar conjuntos de dados pequenos e não ordenados então não teria problema. Apenas não foi utilizada uma lista genérica em Pacientes, pois assume-se que o hospital pode ter centenas ou dezenas de milhares de Pacientes e

para buscar ou verificar Pacientes numa lista genérica demoraria $O(n)$ enquanto em um Dicionário, a busca demoraria $O(1)$ (medida de tempo para algoritmos) , ou seja, uma diferença absurda no final de cada pesquisa (ainda tendo que atender pacientes a todo o momento).

Interfaces

ICrud<T,TId>

Para evitar repetição de código e promover separação entre camadas, foi criada a interface genérica ICrud<T, TId>.

A interface ICrud define oito métodos essenciais para gestão de dados:

- Listar() - Retorna todas as entidades
- JaExiste() - Verifica existência (por entidade ou ID)
- BuscarPorId() - Obtém entidade específica
- Inserir() - Adiciona nova entidade
- Remover() - Remove entidade (por objeto ou ID)
- Atualizar() - Modifica entidade existente

Esta Interface tem umas características que ajudam no desenvolvimento do código, ou seja, é genérica (funciona com qualquer tipo e identificador) e é completa pois define todo o CRUD necessário.

Aplicação no Projeto

Todos os gestores que utilizem Listas implementam esta interface:

GestorMedicoBD → ICrud<Medico, int> (ID)

GestorConsultaBD → ICrud<Consulta, int> (ID)

Comparable<T>

Utilizou-se também a Interface Comparable<T> para poder sortear listas com o método de origem das Estruturas Genéricas nomeadamente *.sort()*. Esta foi utilizada em todas as classes como se fosse um requisito.

O que ela faz é obriga a utilização da função *CompareTo(T? other)* onde se escolhe o método de comparar as classes para as posteriormente ordenar se necessário.

Persistência

Para guardar em ficheiros utilizou-se a serialização binária para garantir que dados não se perdem entre execuções.

Implementação

Classe genérica Ficheiro<T> com dois métodos:

- Guardar(): Serializa lista de objetos em ficheiro binário. Retorna true/false.
- Ler(): Deserializa ficheiro. Se não existir, retorna lista vazia. Em caso de erro, retorna null.

Utiliza BinaryFormatter e FileStream (aviso SYSLIB0011 suprimido por ser funcional para fins académicos).

A persistência é totalmente transparente - camadas superiores não sabem como ou quando dados são guardados. Além disso é tolerante a falhas pois qualquer erro que aconteça o programa não crasha por isso.

É uma gestão de ficheiros genérica, ou seja, funciona com qualquer tipo serializável, esta escolha foi também para poder aprofundar mais um pouco o conceito de Classes Genéricas.

O único problema está em possuir o BinaryFormatter que é obsoleto (mas ainda funcional) mas que para efeito académicos funciona.

Testes Unitários

Estes testes foram implementados com framework NUnit.

O projeto de Testes, encontram-se dois ficheiros:

- TesteNif.cs: Valida comportamento da validação de NIF
- GestorPacienteBDTests.cs: Testa operações CRUD do gestor de pacientes

Testes Implementados

Validação de NIF (4 testes)

- NIF válido (9 dígitos) - não lança exceção
- NIF com letras - lança NifInvalidoException
- NIF null - lança NifInvalidoException
- NIF tamanho incorreto - lança NifInvalidoException

Gestor de Pacientes (4 testes)

- Inserção duplicada - primeira retorna true, segunda false
- Obtenção por NIF - valida tipos Paciente e PacienteBD
- Remoção inexistente - retorna false
- Remoção existente - retorna true

Exceções

Nesta área do trabalho foram implementadas exceções especializadas para serem lançadas ao longo do código, evitando assim confusão caso o programa encontre erros e proporcionando mensagens mais claras e específicas sobre a natureza do problema.

Hierarquia de Exceções

Foi criada uma hierarquia de exceções personalizadas que herdam de exceções personalizadas por exemplo, herda de *ArgumentException*, *KeyNotFoundException*, permitindo tratamento específico para cada tipo de erro do sistema hospitalar que necessitam de exceções mais específicas como *NifInvalidoException*, *EmailInvalidoException*, *EntidadeDuplicadaException*, entre outras.

A utilização de exceções personalizadas traz vários benefícios ao sistema, permitindo identificar imediatamente a origem do erro através do tipo de exceção lançada, facilitando a manutenção do código. Segundo, possibilita mensagens de erro mais descritivas, o que melhora a experiência do utilizador ao fornecer informações claras sobre o que falhou e potencialmente como corrigir.

Funcionalidade Geral

Para melhor compreensão da necessidade de cada classe, observa-se na Figura 1 - Diagrama de Classes do projeto[1] as suas ligações.

A classe *Pessoa* é a classe principal mas uma classe abstrata onde as que herdam de si eram reutilizar os seus atributos, em que as classes que herdam da mesma é a classe *Paciente*, que serve para guardar e gerir os pacientes do hospital. Outra classe que herda de *Pessoa* é *Funcionario*, outra classe abstrata que serve para ser herdada por: *Auxiliar*, *Medico* e *Enfermeiro*. A classe *Auxiliar* serve para guardar auxiliares (Ex: Limpeza, Informatica etc), a classe Médico conecta-se a *Consulta* (uma consulta pode ter um médico mas um médico pode ter várias consultas) e a classe Enfermeiro conecta-se a *EnfermagemCuidados*, que serve para fazer observações de serviços prestados a pessoas internadas no hospital. Uma consulta resulta em vários diagnósticos(classe *Diagnostico*) e vários exames e cada *Exame* possui um resultado. Cada *Internamento* possui um paciente e uma *Cama* que pertence a um *Quarto*.

Fluxo de Funcionamento do Sistema

O programa executável interage exclusivamente com a camada de Regras, que por sua vez comunica com a camada de Dados. Quando o utilizador inicia a aplicação, os gestores da camada Dados gerem todos os dados persistidos em ficheiros. O utilizador pode então realizar operações como registar novos pacientes, marcar consultas, registar internamentos ou consultar históricos médicos através da consola.

Ao registar um novo paciente, o sistema valida primeiro todos os dados através da camada de Regras (verifica se o NIF é válido, se o paciente é menor de idade para pediatria, se não existe duplicado), e só então persiste a informação através do GestorPacienteBD na camada de Dados. Este gestor adiciona o paciente ao Dictionary. Para marcar uma consulta, o sistema verifica a disponibilidade do médico, valida a existência do paciente, cria o objeto Consulta e adiciona-o tanto ao histórico do paciente (na classe PacienteBD) como à lista geral de consultas. Durante a consulta,

o médico pode adicionar múltiplos diagnósticos e solicitar vários exames, sendo todos estes elementos persistidos automaticamente.

Todas estas operações respeitam a arquitetura N-Tier implementada, onde cada camada tem responsabilidades bem definidas: o Programa apenas apresenta informação e recolhe inputs, a camada Regras aplica validações e regras de negócio, e a camada Dados gere persistência e estruturas de dados. Esta separação facilita a manutenção e permite alterações numa camada sem afetar as outras, cumprindo os princípios SOLID especialmente o Single Responsibility e Dependency Inversion.

Repositório GitHub:

[Repositório GitHub](#)

Conclusão

Este projeto representou uma aplicação prática e integrada dos conceitos fundamentais da Programação Orientada a Objetos, demonstrando como princípios como encapsulamento, herança, polimorfismo e abstração podem ser traduzidos em soluções reais e funcionais. A modelação de um sistema de gestão hospitalar exigiu não só a definição rigorosa de classes e relações, mas também uma reflexão constante sobre a eficiência, modularidade e escalabilidade do código.

A segunda fase do projeto permitiu aprofundar significativamente estes conceitos, introduzindo estruturas de dados adequadas, arquitetura em camadas e boas práticas de engenharia de software. A transição de relações 1:1 para 1:N obrigou a repensar a organização do código, resultando numa arquitetura N-Tier robusta que separa claramente responsabilidades entre as camadas de apresentação, lógica de negócio e acesso a dados. A aplicação dos princípios SOLID garantiu um código mais manutenível e extensível, enquanto o princípio DRY eliminou duplicações através da criação de interfaces genéricas e classes utilitárias reutilizáveis.

A implementação de persistência de dados através de serialização binária assegurou que o sistema mantém o seu estado entre execuções, transformando uma aplicação conceptual numa solução funcional e prática. A escolha criteriosa de estruturas de dados, utilizando Dictionary para pacientes devido ao volume esperado e List para outras entidades com volumes menores, demonstrou compreensão não apenas de sintaxe mas de análise de requisitos e otimização de performance. A criação de testes unitários com NUnit reforçou a qualidade do código, permitindo detetar bugs e aumentando a confiança em futuras mudanças.

Ao longo do desenvolvimento, consolidaram-se competências técnicas essenciais como o uso de interfaces para inversão de dependências (SOLID-D), a criação de classes genéricas para reutilização de código, e a implementação de validações robustas através de exceções personalizadas. Mais importante ainda, reforçou-se a capacidade de resolver problemas complexos com soluções simples, limpas e bem estruturadas, sempre considerando o contexto real de aplicação e as necessidades do sistema.

Em suma, este trabalho não só cumpriu os objetivos da unidade curricular para ambas as fases, como demonstrou a evolução de uma modelação básica de classes para um sistema completo e funcional.

Diogo Alexandre Alves Silva nº31504 Projeto-1ª Fase

A experiência adquirida serviu como base sólida para o desenvolvimento de projetos futuros mais avançados, onde a qualidade do código, a arquitetura bem definida e a correta aplicação dos paradigmas de programação serão ainda mais cruciais. O projeto mostrou também que programação orientada a objetos não é apenas sobre escrever código que funciona, mas sobre criar soluções manuteníveis e escaláveis que resistem ao teste do tempo e às inevitáveis mudanças de requisitos.

Referências

Imagem do diagrama (Figura 1 - Diagrama de Classes do projeto[1]) gerada através do site : [PlantUml](https://plantuml.com/pt-br)

C# : [Wikipédia, 15-11-2025](#)

Abstração : [Rocketseat, 2025](#)

Classe: [Wikipédia, 2025](#)

Programação orientada a objetos: [Wikipédia 2025](#)

SOLID: [Medium,2025](#)